# Combining Survey Encoding and Gradient Boosting for Coronary Heart Disease Classification

Nour Chebil Ismail Moudden Ziyad Boudalal
CouscousAI — CS-433 Machine Learning, EPFL, Fall 2026

Abstract—We predict self-reported coronary heart disease from incomplete health-survey answers. Our NumPy-only system combines histogram-based boosted trees with regularized logistic regression trained by Newton's method. A common three-fold evaluation compares interactions, nonlinear trees and probability blending. The delivered blend obtains development F1 of $0.4436 \pm 0.0036$. We report the exact preprocessing and decision rule, distinguish development estimates from independent evaluation, and provide saved weights and a reproducible prediction script.

## I. Task and Data

The provided BRFSS dataset contains 328 135 labeled respondents, 109 379 test respondents and 321 input variables. The positive class represents a reported heart attack or coronary heart disease diagnosis, rather than a future clinical event. Only 28 975 training examples (8.83%) are positive: always predicting the negative class gives 91.17% accuracy but F1 of zero. We therefore evaluate positive-class F1, $2TP/(2TP + FP + FN)$.

Missing entries account for 44.8% of the training matrix, and 147 variables are missing in more than half the rows. Survey codes also need interpretation: for example, PHYSHLTH=88 means no unhealthy days, whereas 77 and 99 indicate nonresponses [1]. We use the supplied data only.

## II. Preprocessing and Models

### A. Logistic features

The logistic encoder excludes constant variables, columns with over 80% missing entries and eight bookkeeping fields: dates, months, year, SEQNO, _PSU and _STSTR. Column-specific rules map known nonresponses to missing, "none" codes to zero and coded frequencies, height and weight to numerical units. Other columns use the recorded nonresponse-code heuristic.

Variables with at most 30 distinct observed values are one-hot encoded; _STATE is always categorical. Levels observed fewer than 100 times share an "other" category with missing values when present in training; unseen values use this category if available, otherwise the reference level. The most frequent level is the reference. Continuous variables are median-imputed, clipped to training percentiles 1 and 99, and standardized. We add distinct missingness flags, including flags for dropped columns with up to 99% missingness, and indicators of original "none" codes. Finally, we standardize squares of the ten continuous columns most correlated with the target and pairwise products of the 25 most correlated encoded columns. Every statistic and feature selection is learned within the training fold.

### B. Regularized logistic regression

For $y_i \in \{0, 1\}$ and encoded $x_i$, we minimize

$$L(w) = \frac{1}{n} \sum_i [\log(1 + e^{x_i^\top w}) - y_i x_i^\top w] + 10^{-4} \|w\|_2^2. \quad (1)$$

The delivered implementation penalizes all coefficients, including the intercept. Newton updates solve the Hessian system, with backtracking and a maximum of 15 iterations. The Hessian is accumulated in blocks of 20 000 rows. Loss calculations are numerically stable. The six required course methods remain available separately in implementations.py.

### C. Boosted trees and final blend

Our implementation follows gradient boosting [2], using first and second derivatives of logistic loss. Each feature is discretized into at most 63 bins plus a missing bin; splits learn the direction of missing values. Trees have depth four, learning rate 0.1, leaf regularization 1 and minimum child Hessian sum 5. Each tree samples half the training rows and half the features (seed 42). Leaf updates use $-G/(H + 1)$.

The final decision combines the two probabilities:

$$p = 0.7 p_{\text{trees}} + 0.3 p_{\text{logistic}},$$
$$\hat{y} = \begin{cases} +1 & p \geq 0.225, \\ -1 & \text{otherwise.} \end{cases} \quad (2)$$

We refit on all labeled rows with 209 trees and the logistic encoding above. The experiment below reproduces the fold-wise development procedure used to select this configuration.

## III. Evaluation and Results

### A. Common protocol

All rows in Table I use the same three shuffled folds (NumPy RandomState, seed 42; not stratified).

Table I
Recomputed development results on identical folds. Newton uses the final encoding; products are its selected interactions.

| Model | F1 (mean ± SD) | Threshold |
|---|---|---|
| Ridge, raw | $0.4092 \pm 0.0021$ | 0.220 |
| Newton | $0.4302 \pm 0.0028$ | 0.195 |
| Newton + products | $0.4311 \pm 0.0031$ | 0.205 |
| Boosted trees | $0.4421 \pm 0.0035$ | 0.220 |
| Blend (delivered) | $0.4436 \pm 0.0036$ | 0.225 |

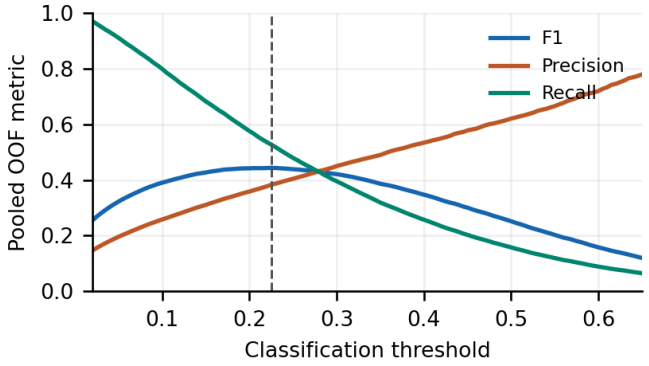



Figure 1. Final blend on pooled development OOF predictions. The dashed line is the delivered threshold 0.225. This curve was used for selection and is not an independent test-set evaluation.

Encoders and model weights are refitted inside each training fold. For tree early stopping, 10% of that fold is held out; training stops after 50 iterations without a validation-loss improvement. The original retained tree counts were 183, 171 and 216; their mean multiplied by 1.1 gives 209 trees for the full-data refit.

We pool out-of-fold (OOF) probabilities and search thresholds from 0.02 to 0.98 in steps of 0.005. The delivered blend uses its fixed threshold 0.225. Reported uncertainty is the population standard deviation of the three fold F1 scores, not a confidence interval. Model settings, the blend weight and threshold were selected on these development data, so these are not unbiased estimates on a final untouched holdout. The raw-feature baseline uses ridge regression ($\lambda = 10^{-4}$), with training-only mean imputation and standardization. Encoded Newton is the linear probabilistic baseline. Adding interactions changes F1 by 0.0009; boosted trees improve over that logistic variant by 0.0110. Blending changes F1 by 0.0015 relative to trees alone. Small changes should be interpreted against fold variability, not as proof of superiority. The tree/logistic comparison also changes the representation; only the interaction and blend ablations isolate a single added component. These results do not establish a universally optimal category cutoff. At threshold 0.5, pooled OOF F1 is 0.2517. The delivered threshold gives precision 38.3% and recall 52.7%, with 15 272 true positives, 24 602 false positives and 13 703 false negatives. This illustrates the practical tradeoff: increasing recall still leaves substantial false alarms and missed cases.

## IV. Discussion and Reproducibility

Our main finding is that nonlinear trees improve on an encoded logistic model, while blending gives a smaller additional change. Survey coding remains an assumption: heuristic nonresponse detection may misinterpret valid categories. Repeated development-set selection can overestimate generalization. The data are self-reported and cross-sectional; the model is neither a causal analysis nor a validated prospective screening tool.

We recorded an earlier AIcrowd F1 of 0.455 in our experiment log. The original uploaded CSV was unavailable for comparison, so we do not claim that this score has been independently verified for the regenerated file. The repository includes the delivered weights, CSV fingerprints and the full training code. python run.py generates predictions from the saved weights; python run.py –train refits the recorded model. The validation script exports per-fold results and OOF predictions, from which the final figures and table are generated automatically. This keeps the report, configuration and executable submission consistent.

## References

[1] Centers for Disease Control and Prevention, “BRFSS 2015 Codebook,” 2016.

[2] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.